



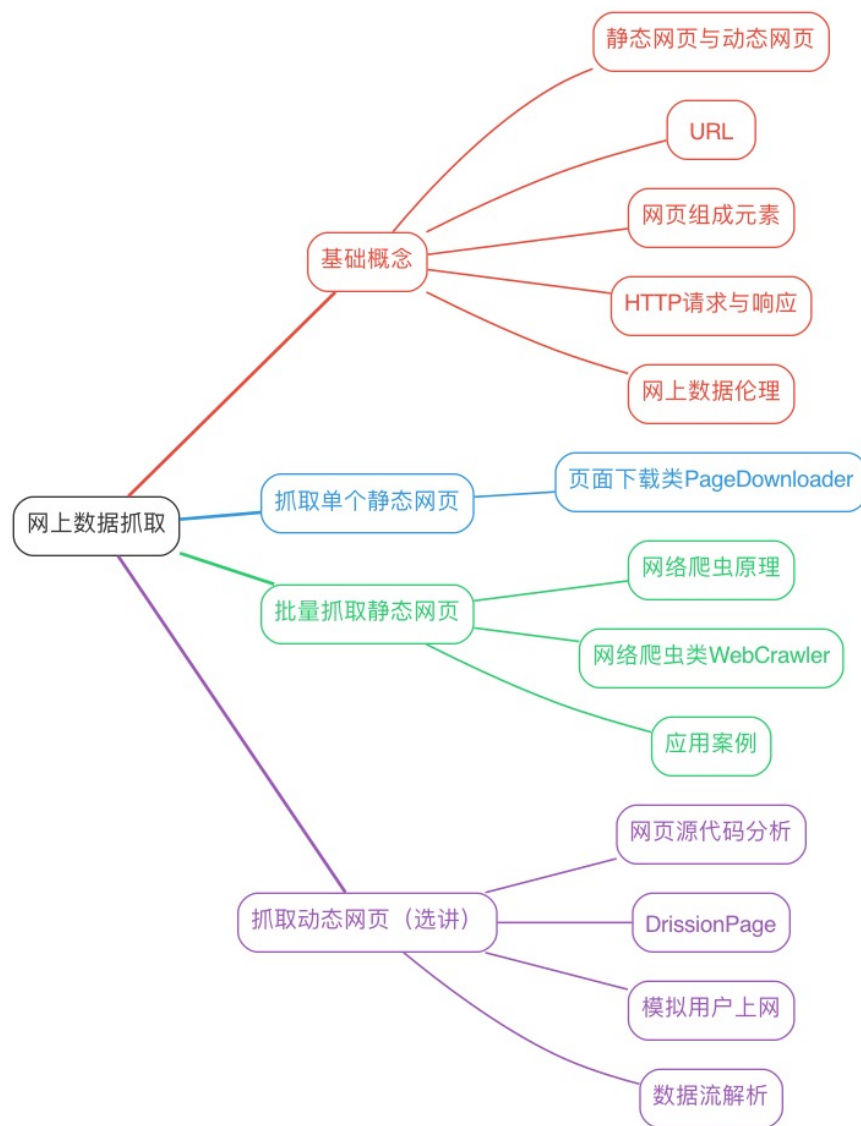
《实用Python编程》

第6讲 网上数据抓取

实用Python编程 教研组



提纲





网上数据的重要性

- 互联网是信息发布与传播的主要渠道
- 互联网上的数据是大数据分析与决策的重要基础
 - 填报高考志愿不仅需要参考绝对分数，还需要省份排名信息
 - 排名信息往往在不同网站上，这就要求能够对网上数据进行批量抓取与汇总
- 访问网站数据涉及向服务器发送**请求**(Request)，以及解读服务器返回的**响应**(Response)

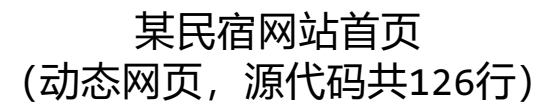
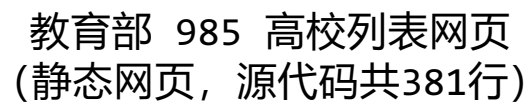
从基础概念入手，介绍如何（批量）抓取（静态）网页数据，并保存为本地文件

基础概念

- 网页 (web page) 是存储在远程服务器上的HTML文件，是互联网上承载数据的核心载体

静态网页	动态网页
内容由开发者预编写	内容由服务器端脚本实时生成
所有元素结构、呈现形式和交互逻辑都包含在HTML文件中	通过JavaScript实现内容呈现和交互功能，HTML作为脚本容器
源码较长	源码较短

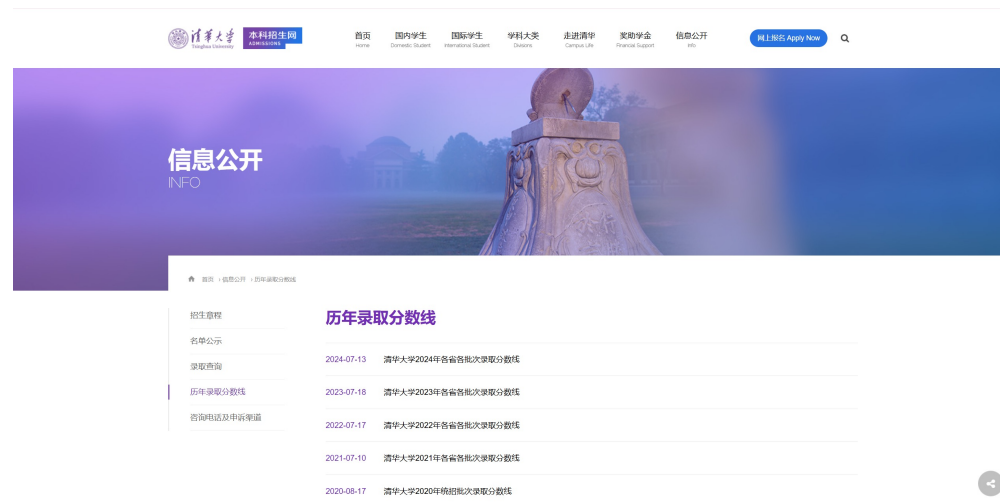
- 网页示例





基础概念

- 统一资源定位符 (Uniform Resource Locator, URL) 是互联网上资源的唯一地址, 也被称为“网址”
 - 与文件路径类似, URL使用斜杠(/)表示目录的层次关系
 - 示例: <https://join-tsinghua.edu.cn/xxgk/lnlqfsx.htm>



将URL转为文件名



通过自定义函数将URL编码为文件名，以便将网页保存到本地

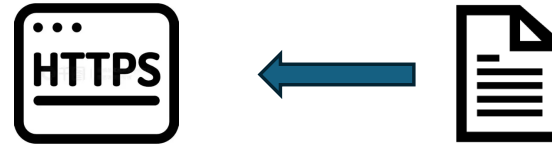
- Step 1. 将URL编码为字节数据: `encode('utf-8')`
- Step 2. 对字节数据进行Base64编码: `urlsafe_b64encode(url_bytes)`
- Step 3. 将Base64字节解码为字符串: `decode('utf-8')`
- Step 4. 移除可能存在的填充字符: `rstrip('=')`

```
import base64

"""将URL编码为安全的文件名"""
def url_to_filename(url):
    url_bytes = url.encode('utf-8')
    base64_bytes = base64.urlsafe_b64encode(url_bytes)
    filename = base64_bytes.decode('utf-8')
    filename = filename.rstrip('=') # 移除可能存在的填充字符 (=)
    return filename
```

不同的URL需要对应不同的文件名。为便于数据溯源，要求能从文件名解码回URL

文件名转为URL

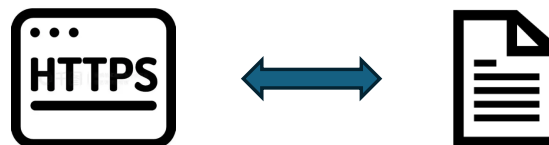


通过自定义函数将文件名解码回原始URL

- Step 1. 文件名预处理，视情况添加填充字符：'='*padding
- Step 2. 将文件名编码为字节数据：encode('utf-8')
- Step 3. 对字节数据进行Base64解码：urlsafe_b64decode(base64_bytes)

```
"""将文件名解码回原始URL"""  
def filename_to_url(filename):  
    padding = 4 - (len(filename) % 4) # 填充字符 (=) 回填  
    if padding != 4:  
        filename += '=' * padding  
    base64_bytes = filename.encode('utf-8')  
    url_bytes = base64.urlsafe_b64decode(base64_bytes)  
    return url_bytes.decode('utf-8')
```

URL与文件名互转



- 测试样例

```
test_urls = '''https://gaokao.eol.cn/bei_jing/dongtai/202407/t20240720_2625168.shtml
https://gaokao.eol.cn/bei_jing/dongtai/202406/t20240625_2619192.shtml
https://www.dxsbb.com/news/46721.html
http://www.moe.gov.cn/srcsite/A22/s7065/200612/t20061206_128833.html
http://www.moe.gov.cn/srcsite/A22/s7065/200512/t20051223_82762.html
https://rdzs.ruc.edu.cn/inquiry/admission/indexcms
https://rdzs.ruc.edu.cn/inquiry/enrollplan/indexcms
https://m.tujia.com/hotel_city73'''
for ori_url in test_urls.split():
    filename = url_to_filename(ori_url)
    new_url = filename_to_url(filename)
    print(f"{ori_url} -> {filename}")
    assert(ori_url == new_url)
```

当assert的表达式不成立时，会
抛出AssertionError异常

assert是调试代码的有效手段



网页组成元素

- **HTML**定义网页的结构和内容
- **CSS**(Cascading Style Sheets, 层叠样式)定义网页中元素的样式
 - 例如, 颜色、尺寸等
- **JavaScript**(JS)赋予网页动态交互能力

第14讲 (Streamlit) 将介绍如何仅用Python编写网页程序



课堂练习-3-2-1

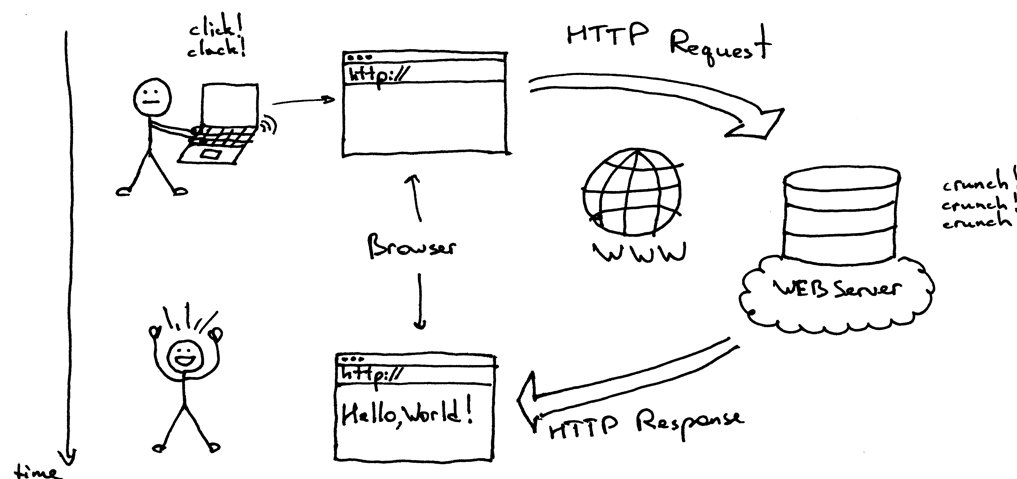
将复旦大学历年录取分数网站的URL转换为文件名，再将文件名还原为URL

- <https://ao.fudan.edu.cn/36333/list.htm>

基础知识

HTTP请求与响应

- **客户端(Client)**: 浏览器或Python程序, 向服务器发送请求
- **服务器(Server)**: 存放网页的远程计算机, 解析请求并发送响应
- **请求(Request)**: 客户端向服务器发送的“指令”, 告知服务器 需要什么内容
- **响应(Response)**: 服务器解析请求后, 返回给客户端的“结果”, 包含网页数据





HTTP请求与响应

- 使用requests库发送HTTP请求
 - `requests.get` 构建一个请求发向服务器
 - 返回`requests.models.Response`对象

安装命令 `pip install requests`

示例代码 3.21 使用 requests 发送 HTTP 请求

```
1 import requests
2
3 target_url = "http://www.moe.gov.cn/srcsite/A22/s7065/200612/t20061206_128833.html"
4 response = requests.get(target_url, headers={"User-Agent": "Mozilla/5.0"})
5 print(response.status_code, response.encoding) # 输出 200 ISO-8859-1
```

“985工程”学校名单 http://www.moe.gov.cn/srcsite/A22/s7065/200612/t20061206_128833.html



HTTP请求与响应

- 解读`requests.get()`的返回结果 `response`
 - `status_code` 状态码为 200 表示访问过程正常，否则访问异常
 - 2xx: success、3xx redirection、4xx client error、5xx server error

表 3.7: 响应对象 `response` 的常用属性

属性	作用
<code>status_code</code>	响应状态码，如果不是 200 说明访问过程出现错误
<code>headers</code>	响应头信息（字典格式）
<code>cookies</code>	服务器返回的 Cookie（字典格式）
<code>encoding</code>	从响应头中获取的响应内容编码方式
<code>apparent_encoding</code>	根据响应内容推断的响应编码方式
<code>text</code>	字符串格式的响应内容
<code>content</code>	二进制格式的响应内容

使用自动检测的编码，以免保存
(中文) 文本时出现乱码

HTTP请求与响应

- 尝试向不存在的URL发送请求
 - 网络通信过程中的不可控因素较多，可能导致通信失败
 - 使用try...except模块处理异常是良好的编程习惯

示例代码 3.22 尝试向不存在的 URL 发送

```
1 try:
2     response = requests.get("http://www.doesnotexist.html")
3     print(response.status_code)
4 except Exception as e:
5     print (e)
```

```
try:
    resp = requests.get('https://www.google.com/', timeout=2)
    print(resp.status_code)
except Exception as e:
    print(e)
```

设置超时时间，
避免请求长时间挂起

基础知识

- 网上数据伦理与Robots协议
 - 网站根目录下的robots.txt文件规定了哪些页面可以抓取
 - 爬虫程序应当严格遵守该协议的约定

表 3.8: Robots 协议组成部分

组成部分	作用	示例
User-agent	指定协议适用的爬虫类型	1 User-agent: Baiduspider # 仅针对百度爬虫 2 User-agent: * # 适用于所有爬虫
Disallow	禁止访问指定路径及子路径	1 Disallow: /private/ # 禁止访问/private目录 2 Disallow: /images/ # 禁止访问/images目录
Allow	允许访问特定路径	1 Allow: /folder1/myfile.html # 允许访问该文件

抓取单个静态网页

- 抓取一个特定静态网页并保存为本地的HTML文件
- 定义一个页面下载类PageDownloader

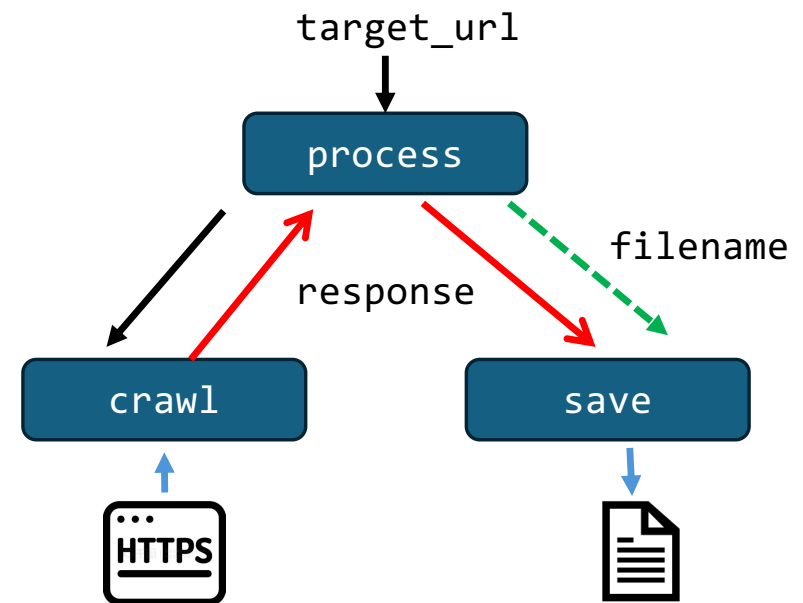
- 属性

- `save_dir` #存放数据的路径

- 方法

- `process(self, target_url)`
 - `crawl(self, url)`
 - `save(self, response, filename)`

类方法之间调用（协作）示意





抓取单个静态网页

- 导入模块`os`, `requests`
- 定义类`PageDownloader`
 - 类属性`save_dir`: 数据存储目录。该属性值可在创建一个`PageDownloader`类对象时指定

```
import os
import requests

class PageDownloader:
    def __init__(self, save_dir="html_pages"):
        self.save_dir = save_dir
        os.makedirs(self.save_dir, exist_ok=True)
```



抓取单个静态网页

- 类方法 `crawl`: 发送 HTTP 请求并获得 `response` 对象
 - `requests.get` 构建并发送请求
 - `raise_for_status` 状态码异常处理

```
def crawl(self, url):  
    try:  
        response = requests.get(url, headers={"User-Agent": "Mozilla/5.0"})  
        response.raise_for_status() # 状态码异常处理  
        return response  
    except requests.exceptions.RequestException as e:  
        print(f"[Error] 获取页面失败: {e}")  
        return None
```

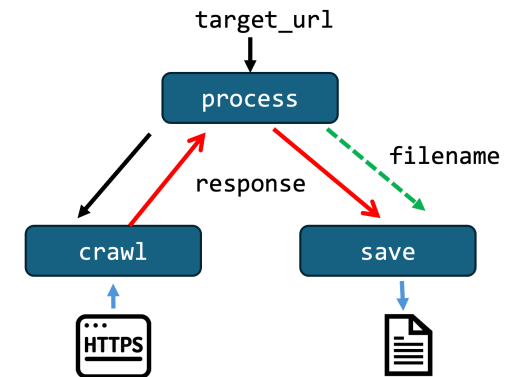
抓取单个静态网页

- 类方法`save`: 将`response`中的文本内容保存到本地HTML文件
- 文件名`filename`通过调用之前的`url_to_filename`函数生成
 - `fw.write(response.text)` 将网页内容写入文件

```
def save(self, response, filename):  
    try:  
        if response.encoding.lower() == 'none': # 检测并设置正确的文本编码  
            response.encoding = response.apparent_encoding  
        with open(filename, 'w', encoding=response.encoding) as fw: # 保存内容  
            fw.write(response.text)  
        print(f"[Status] 已保存到 {filename}")  
        return True  
    except Exception as e:  
        print(f"[Error] 保存页面失败: {e}")  
        return False
```

抓取单个静态网页

- 类方法`process`: 处理当前任务的统一接口
 - `crawl` 发送请求
 - `save` 保存网页内容



```
def process(self, target_url):
    print(f"[Status] 开始抓取 {target_url}")
    response = self.crawl(target_url)
    if response and response.status_code == 200:
        filename = os.path.join(self.save_dir, url_to_filename(target_url) + '.html')
        self.save(response, filename)
    else:
        response = None
        message = "成功" if response else "失败"
        print(f"[Status] 页面抓取{message}")
    return response
```



抓取单个静态网页

- 测试代码
 - `pd = PageDownloader()` 创建PageDownloader类对象
 - `pd.process(test_url)` 调用类方法process

```
"""测试代码"""  
test_url = "http://www.moe.gov.cn/srcsite/A22/s7065/200612/t20061206_128833.html"  
pd = PageDownloader()  
pd.process(test_url)
```



课堂练习-3-2-2

- 用 PageDownloader 抓取复旦大学历年录取分数网页
<https://ao.fudan.edu.cn/36333/list.htm>

批量抓取静态网页

- 互联网是一张以网页为节点的大网，节点之间通过**超链接** (hyperlink)的方式连接
 - HTML源码中标签<a>的href属性指定了超链接

```
<a href="http://www.moe.gov.cn/">
```
- **网络爬虫** (web crawler) 就是一个能够沿着这些“超链接”在互联网上自动“爬行”的程序
 - 它从一个或者多个初始URL出发，抓取页面内容，然后从页面中解析出新的URL，再继续抓取，如此循环

网络爬虫

- 工作过程
 - 爬虫把待处理的`url`放在工作队列`task_queue`中
 - 爬虫依次处理队列中的`url`，顺着网页把其他`url`添加到队列中
 - 当队列为空时，爬虫就会停止

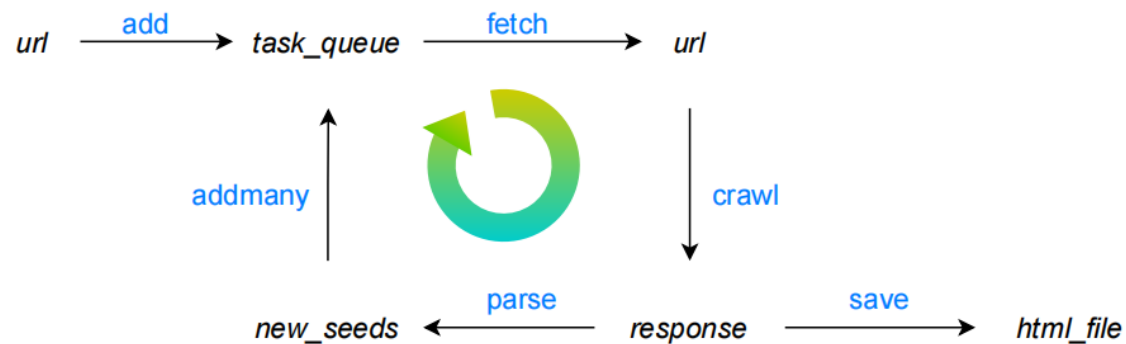


图 3.7: 网络爬虫工作示意图



网络爬虫的实现

- 先导入必要的库

```
import time, random, requests  
from urllib.parse import urlparse, urljoin  
from bs4 import BeautifulSoup
```

urlparse用来拆解和分析URL

urljoin用来拼接和生成完整URL



网络爬虫的实现

定义PageDownloader的一个子类WebCrawler，作为网络爬虫类，避免重复编写底层代码

- save_dir: 网页存储目录
- `super().__init__(save_dir)` # 调用父类初始化
- `self.task_queue = []` # URL工作队列
- `self.url_cache = set()` # 已访问URL集合(去重, 防止重复爬取)

```
class WebCrawler(PageDownloader):  
    def __init__(self, trace_link=True, save_dir="html_pages"):  
        super().__init__(save_dir)  
        self.trace_link = trace_link  
        self.task_queue = []  
        self.url_cache = set() # 已访问 URL 集合, 避免重复爬取
```

网络爬虫的实现

- 重写process方法

- `super().process(url)` # 调用父类方法从指定url中下载html文本
- `self.url_cache.add(url)` # 标记为已访问,避免被再次访问
- `self.parse(response.txt, url)` # 从下载的html文本中提取新链接

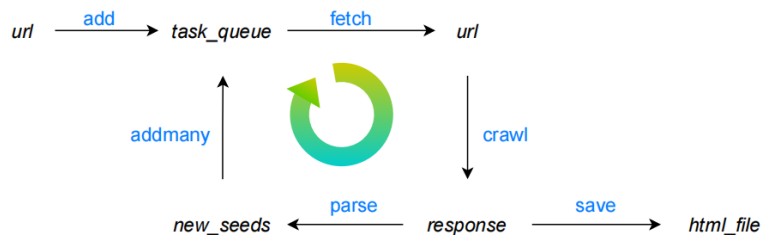


图 3.7: 网络爬虫工作示意图

```
def process(self, url):  
    response = super().process(url)  
    self.url_cache.add(url)  
    if response and self.trace_link:  
        new_seeds = self.parse(response.text, url)  
        self.addmany(new_seeds)  
    return response
```

网络爬虫的实现

- parse方法：从指定的HTML文本中解析出所有有效超链接
 - BeautifulSoup(htmlstr, 'html.parser') 创建html解析器
 - for a_tag in soup.find_all 循环遍历所有<a>标签

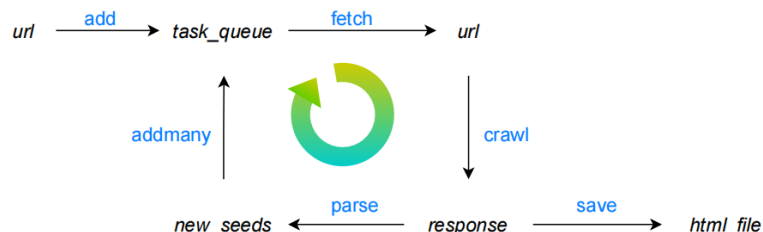


图 3.7: 网络爬虫工作示意图

```
def parse(self, htmlstr, base_url):
    soup = BeautifulSoup(htmlstr, 'html.parser')
    new_links = set()
    for a_tag in soup.find_all('a', href=True):
        href = a_tag['href'].strip()
        if not href or href.startswith(('javascript:', 'mailto:', 'tel:')): # 跳过无效链接
            continue
        link = urljoin(base_url, href) # 规范化URL (转换为绝对路径并移除片段标识符)
        parsed = urlparse(link)
        if parsed.fragment:
            link = parsed._replace(fragment='').geturl()
        new_links.add(link)
    return list(new_links)
```

网络爬虫的实现

- 将一个或多个URL加入到工作队列中
 - `add(self, url)` 向队列中添加一个没有访问过的URL
 - `addmany(self, urls)` 批量添加URL

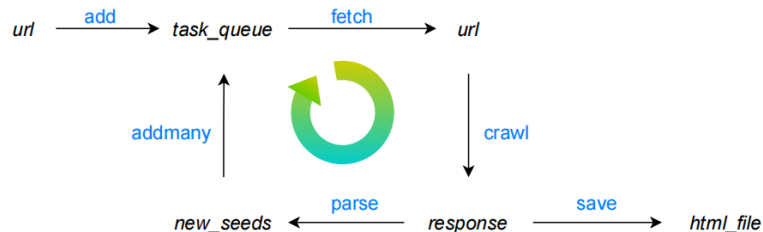


图 3.7: 网络爬虫工作示意图

```
def add(self, url):  
    if url not in self.url_cache:  
        self.task_queue.append(url)  
  
def addmany(self, urls):  
    for url in urls:  
        self.add(url)
```

添加新任务到队列尾部

网络爬虫的实现

- 从工作队列中取出一个URL
 - `self.task_queue.pop(0)` 取出队列的第一个元素

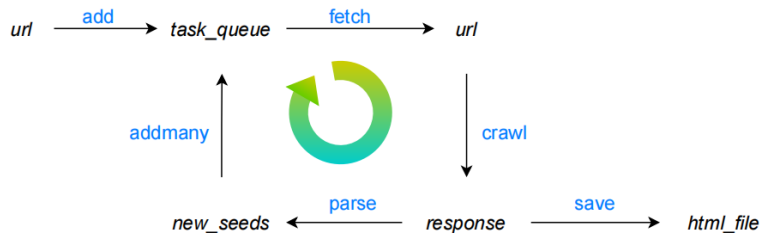


图 3.7: 网络爬虫工作示意图

```
def fetch(self):  
    print(f"[Status] 当前工作队列长度: {len(self.task_queue)}")  
    if self.task_queue:  
        return self.task_queue.pop(0)  
    else:  
        return None
```

网络爬虫的实现

- 爬虫程序统一入口
 - url = self.fetch() 获取第一个URL
 - while url 循环直到队列为空或达到停止条件
 - time.sleep(sleep_time) 随机休眠，以免影响网站正常运行

```
def run(self):
    url = self.fetch()
    while url:
        res = self.process(url)
        if len(self.url_cache) > 10: # 抓了10个网页后就强制结束
            break
        sleep_time = random.uniform(1, 5)
        print(f"[Status] Sleeping for {sleep_time:.2f} seconds...")
        time.sleep(sleep_time) # 随机休眠，以免影响网站正常运行
        url = self.fetch()
    print(f"[Status] {self.__class__.__name__} stopped after {len(self.url_cache)} pages visited.")
```



网络爬虫的实现

- 测试代码

```
test_url = "http://www.moe.gov.cn/srcsite/A22/s7065/200612/t20061206_128833.html"
wc = WebCrawler() # 创建网页爬虫类对象
wc.add(test_url)  # 向工作队列添加初始URL
wc.run()          # 启动爬虫程序
```



应用案例：抓取清华大学历年录取分数

- 网址 <https://join-tsinghua.edu.cn/xxgk/lnlqfsx.htm>
- 先观察网页内容
 - 在网页空白处点击鼠标右键，在小窗列表中选中“查看网页源代码”，可见年份、链接等信息均置于如下所示的HTML超链接标签<a>中，且格式较为规范

历年录取分数线

	div.name	827.33 × 56.76
2024-07-13	清华大学2024年各省各批次录取分数线	
2023-07-18	清华大学2023年各省各批次录取分数线	
2022-07-17	清华大学2022年各省各批次录取分数线	
2021-07-10	清华大学2021年各省各批次录取分数线	
2020-08-17	清华大学2020年统招批次录取分数线	
2019-07-11	清华大学2019年各省各批次录取分数线	

```
<div>
  <ul>
    <li>
      <a href=".../info/1032/1945.htm" title="清华大学2024年各省各批次录取分数线">
        <div class="time">2024-07-13</div>
        ...
        <div class="name">清华大学2024年各省各批次录取分数线</div> == $0
      </a>
    </li>
    <li>
      <a href=".../info/1032/1808.htm" title="清华大学2023年各省各批次录取分数线">
        <div class="time">2023-07-18</div>
        <div class="name">清华大学2023年各省各批次录取分数线</div>
      </a>
    </li>
    <li>
      <a href=".../info/1032/1683.htm" title="清华大学2022年各省各批次录取分数线">
        ... </a>
      </li>
    <li>
      <a href=".../info/1032/1506.htm" title="清华大学2021年各省各批次录取分数线">
        ... </a>
      </li>
    <li>
      <a href=".../info/1032/1157.htm" title="清华大学2020年统招批次录取分数线">
```



应用案例：抓取清华大学历年录取分数

- Step 1. 抓取目录页

```
1 base_url = "https://join-tsinghua.edu.cn/xxgk/lnlqfsx.htm"
2 wc = WebCrawler(trace_link=False, save_dir='thu')
3 response = wc.process(base_url)
4 response.encoding = response.apparent_encoding # 使用自动检测的编码, 以免文本出现乱码
```

注意参数trace_link设为False

```
[Status] 开始抓取 https://join-tsinghua.edu.cn/xxgk/lnlqfsx.htm
[Status] 已保存到 thu/aHR0cHM6Ly9qb2luLXRzaW5naHVhLmVkdS5jb3R0eGdrL2xubHFMc3guaHRt.html
[Status] 页面抓取成功
```

- Step 2. 直接从目录页中提取历年数据网址
- Step 3. 再次调用wc, 批量下载



应用案例：抓取清华大学历年录取分数

- Step 2. 直接从目录页中提取历年数据网址

```
<div>
  <ul>
    <li>
      <a href="._/info/1032/1945.htm" title="清华大学2024年各省各批次录取分数线">
        <div class="time">2024-07-13</div>
        <div class="name">清华大学2024年各省各批次录取分数线</div> == $0
      </a>
    </li>
    <li>
      <a href="._/info/1032/1808.htm" title="清华大学2023年各省各批次录取分数线">
        <div class="time">2023-07-18</div>
        <div class="name">清华大学2023年各省各批次录取分数线</div>
      </a>
    </li>
    <li>
      <a href="._/info/1032/1683.htm" title="清华大学2022年各省各批次录取分数线">
        ... </a>
      </li>
    <li>
      <a href="._/info/1032/1506.htm" title="清华大学2021年各省各批次录取分数线">
        ... </a>
      </li>
    <li>
      <a href="._/info/1032/1157.htm" title="清华大学2020年统招批次录取分数线">
        ... </a>
      </li>
  </ul>
</div>
```

```
1 years = [2021, 2022, 2023, 2024]
2 keywords = [f"{year}年" for year in years]
3 soup = BeautifulSoup(response.text, "html.parser")
4 url_list = []
5 for a_tag in soup.find_all("a", href=True):
6     text = a_tag.get_text(strip=True)
7     if any(keyword in text for keyword in keywords):
8         new_url = urljoin(base_url, a_tag["href"])
9         url_list.append(new_url)
10 print("\n".join(url_list))
```

```
https://join-tsinghua.edu.cn/info/1032/1945.htm
https://join-tsinghua.edu.cn/info/1032/1808.htm
https://join-tsinghua.edu.cn/info/1032/1683.htm
https://join-tsinghua.edu.cn/info/1032/1506.htm
```



应用案例：抓取清华大学历年录取分数

• Step 3. 再次调用wc, 批量下载

```
1 wc.addmany(url_list)
2 wc.run()
```

```
[Status] 当前工作队列长度: 4
[Status] 开始抓取 https://join-tsinghua.edu.cn/info/1032/1945.htm
[Status] 已保存到 thu/aHR0cHM6Ly9qb2luLXRzaW5naHVhLmVkdS5jb9pbmZvLzEwMzIvMTk0NS5odG0.html
[Status] 页面抓取成功
[Status] Sleeping for 4.07 seconds...
[Status] 当前工作队列长度: 3
[Status] 开始抓取 https://join-tsinghua.edu.cn/info/1032/1808.htm
[Status] 已保存到 thu/aHR0cHM6Ly9qb2luLXRzaW5naHVhLmVkdS5jb9pbmZvLzEwMzIvMTgwOC5odG0.html
[Status] 页面抓取成功
[Status] Sleeping for 3.17 seconds...
[Status] 当前工作队列长度: 2
[Status] 开始抓取 https://join-tsinghua.edu.cn/info/1032/1683.htm
[Status] 已保存到 thu/aHR0cHM6Ly9qb2luLXRzaW5naHVhLmVkdS5jb9pbmZvLzEwMzIvMTY4My5odG0.html
[Status] 页面抓取成功
[Status] Sleeping for 3.78 seconds...
[Status] 当前工作队列长度: 1
[Status] 开始抓取 https://join-tsinghua.edu.cn/info/1032/1506.htm
[Status] 已保存到 thu/aHR0cHM6Ly9qb2luLXRzaW5naHVhLmVkdS5jb9pbmZvLzEwMzIvMTUwNi5odG0.html
[Status] 页面抓取成功
[Status] WebCrawler stopped after 5 pages visited.
```



课堂练习-3-2-3

利用WebCrawler抓取复旦大学近5年录取分数数据，并保存到名为fdu的文件夹中。
目录页 <https://ao.fudan.edu.cn/36333/list.htm>

ao.fudan.edu.cn/36333/list.htm	
报考指南	录取分数
招生政策	
招生动态	
招生计划	
录取分数	
	复旦大学2025年分省录取分数
	复旦大学2024年分省录取分数
	复旦大学2023年分省录取分数
	复旦大学2022年分省录取分数
	复旦大学2021年分省录取分数
	复旦大学2020年分省录取分数
	复旦大学2019年分省录取分数
	复旦大学2018年分省录取分数
	复旦大学2017年分省录取分数
	复旦大学2016年分省录取分数
	复旦大学2015年分省录取分数

当前位置: 首页 | 报考

每页 14 记录 总共 11 记录 第一页 << 上一页 下一页 >> 尾页 页码 1/1 [



进阶内容：抓取动态网页（选讲）

- 动态网页的抓取难度要高于静态网页
 - 动态网页的内容往往由JavaScript代码实时生成，不能像处理静态网页那样先抓取后解析
 - 一般要求用户登录之后方可浏览相关网页
- 需要解决两个问题
 - 用户登录
 - 基于实时HTML数据流的信息抽取

以民宿数据抓取为背景，介绍如何使用DrissionPage库解决上述两个问题



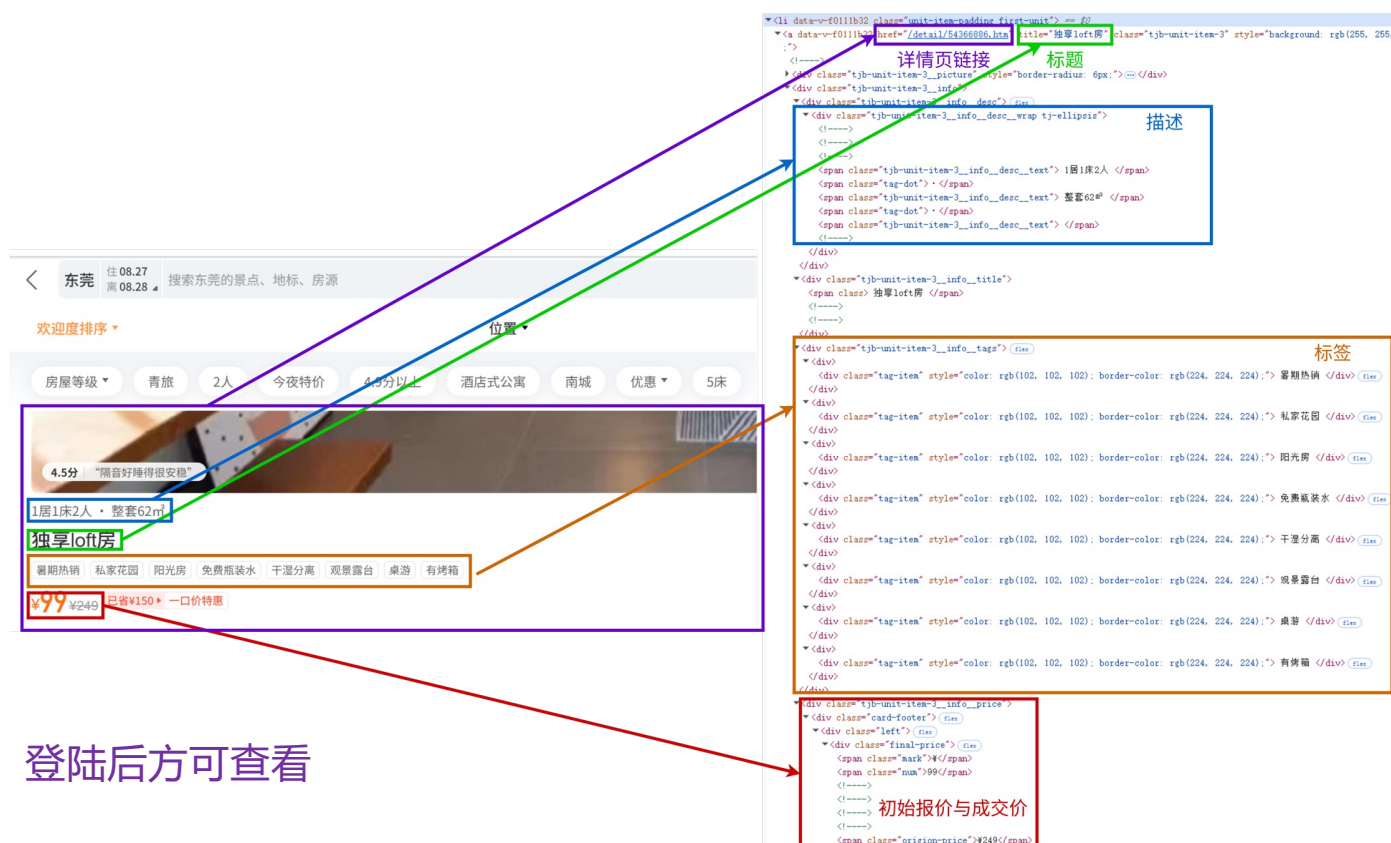
使用DrissionPage抓取动态网页

- 本节使用的教学数据来自某民宿平台 (<https://m.tujia.com>) 需注册登陆
- 在该平台上, 每个城市都有一个包含该城市编号的专有URL
 - `https://m.tujia.com/hotel_city[城市编号]`
 - 比如东莞的编号为73 "`https://m.tujia.com/hotel_city73`"
- 在URL中可以设置价格参数对民宿进行筛选。
 - `https://m.tujia.com/hotel_city[城市编号]/7-[价格下限],[价格上限]`
 - 比如"`/hotel_city73/7-2,101/`"会列出东莞所有价格介于2~101元之间的民宿
- 拟抓取的目标数据: 民宿画像(位置、户型、面积、朝向、装修、报价、成交价等)

本质上是利用DrissionPage来模拟用户的上网行为, 包括打开浏览器、输入网址、浏览网页、翻页等

先分析网页源代码

- 为了抓取目标数据，我们需要借助浏览器自带的“开发者工具”观察网页源代码，以定位相关数据



详情页链接

标题

描述

标签

初始报价与成交价

登陆后方可查看

可借助大模型来完成网页源代码分析工作

先分析网页源代码

- 表3.9总结了该民宿网页种各项数据与HTML/CSS元素的对应关系
- 以 CSS 类名为关键词就可以在HTML数据流中定位相应的数据项

表 3.9 在动态网页中根据 HTML/CSS 元素定位各项数据

目标数据	对应 HTML/ CSS 元素	Python 语句
首个民宿	class="unit-item-padding"	item=bro.ele(".unit-item-padding")
民宿列表	同上	items=bro.eles(".unit-item-padding")
民宿链接	<a>	item_link=item.ele("tag:a")
标题	<a> 中的 title 属性	title=item_link.attr("title")
详情页链接	<a> 中的 href 属性	href=item_link.attr("href")
标签	class="tjb-unit-item-3__info__tags"	tags=item.ele(".tjb-unit-item-3__info__tags").text
描述	class="tjb-unit-item-3__info__desc"	desp=item.ele(".tjb-unit-item-3__info__desc").text
报价	class="origion-price"	ori_price=item.ele(".origion-price").text
成交价	class="fnal-price"	fhspcace {0.1mm}inal_price=item.ele(".final-price").text

注：变量 bro 表示DrissionPage 中的 ChromiumPage对象。



DrissionPage模拟用户上网

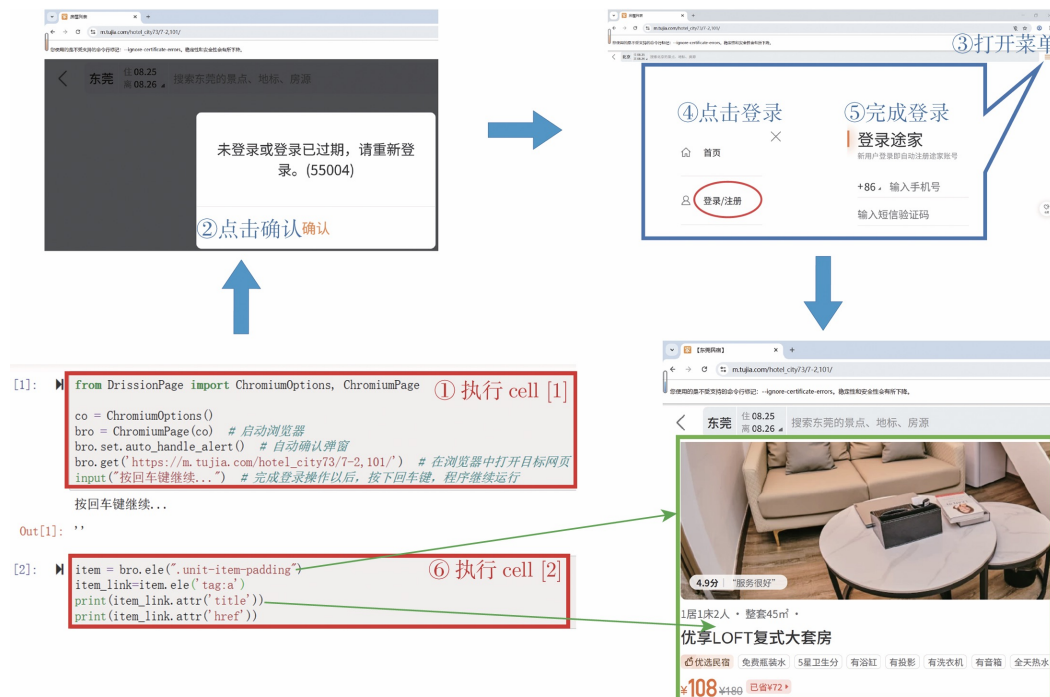
第一步，启动浏览器。通过创建一个ChromiumPage类对象bro来启动一个浏览器

```
from DrissionPage import ChromiumOptions, ChromiumPage
co = ChromiumOptions()
bro = ChromiumPage(co)
bro.set.auto_handle_alert() # 自动确认弹窗，避免弹窗影响网页内容抓取
```

DrissionPage模拟用户上网

第二步，向目标URL请求网页。通过bro.get(target_url)在前一步启动的浏览器中打开目标网页。

```
bro.get("https://m.tujia.com/hotel_city73/7-2,101/")
input("按回车键继续...") # 完成登录操作以后，按下回车键，程序继续运行
```



注意：民宿网站的数据需要在用户登录后才能在页面显示。



从实时HTML数据流中提取信息

- 解析网页并提取所需信息
 - 与BeautifulSoup的find与find_all类似, DrissionPage提供了ele与eles方法, 分别用于定位单个目标元素和所有目标元素。

```
item = bro.ele(".unit-item-padding")
item_link = item.ele("tag:a")
print(item_link.attr("title")) # 获取标题
print(item_link.attr("href")) # 获取详情页链接
```

从实时HTML数据流中提取信息

数据抓取策略:

- 首先, 利用`items=bro.eles(".unit-item-padding")`获取民宿列表
- 然后, 遍历每个民宿对象获取各个数据字段
- 对于少量房源部分字段缺失的情况, 我们以`try...except`的方式进行处理

```
1 def get_one_page(bro):
2     records = []
3     items = bro.eles(".unit-item-padding")
4     for item in items:
5         try:
6             item_link = item.ele("tag:a")
7             title = item_link.attr("title")
8             href = item_link.attr("href")
9             tags = item.ele(".tjb-unit-item-3__info__tags").text
10            desp = item.ele(".tjb-unit-item-3__info__desc").text
11            ori_price = item.ele(".origion-price").text
12            final_price = item.ele(".final-price").text
13            records.append((title, href, tags, desp, ori_price, final_price))
14        except Exception as e:
15            print(f"[Error] {e}")
16    return records
```

完整方案

```
1 co = ChromiumOptions()
2 bro = ChromiumPage(co) #启动浏览器
3 bro.set.auto_handle_alert() #自动确认弹窗
4 bro.get("https://m.tujia.com/hotel_city73/7-2,101/") #在浏览器中打开目标网页
5 input("按回车键继续...") #待登录完成后, 按回车键继续
6
7 item_list = [] #房源列表
8 max_pages = 10 #待抓取页数上限
9 start_time = time.time()
10
11 for page_idx in range(max_pages):
12     res = get_one_page(bro)
13     if len(res) == 0: #没有抓到新数据, 提前结束
14         break
15     item_list += res
16     time.sleep(5) #每抓取一个页面后暂停5秒, 防止触发反爬机制被封禁
17     if time.time()-start_time > 300: #每满5分钟, 程序额外休眠1分钟
18         time.sleep(60)
19         start_time = time.time() #重置起始时间
20     bro.scroll.to_bottom() #模拟页面滚动到底部, 实现自动翻页
21
22 #使用Pandas保存为CSV
23 df = pd.DataFrame(item_list)
24 df.drop_duplicates(keep='first', inplace=True) #去重: 对于重复数据, 只保留该数据的首次记录
25 df.to_csv("minsu-overview.csv", index=False, encoding="utf-8-sig") #指定utf-8-sig编码避免乱码
```

抓取一段时间后进行休眠以防止触发反爬机制被封禁